

NestShip Frontend Roadmap

- Planned repository: `nestship-frontend`
- Team ownership: Frontend team
- Baseline date: 2026-08-07
- Current status: Repository not yet represented in this backend workspace
- Backend contract source: NestShip Swagger/OpenAPI under `/api/docs-json`

Purpose

This document contains only frontend-owned work for NestShip. It preserves the phase numbers from the original project plan, so numbering is intentionally non-contiguous where a phase is owned entirely by the backend.

The frontend owns browser architecture, user journeys, presentation, accessibility, client-side state, API consumption, responsive behavior, and frontend testing. Backend authorization, tenant isolation, billing entitlements, and security decisions remain authoritative even when the frontend hides or disables an action.

Status Legend

Status	Meaning
Complete	Implemented, tested, documented, committed, and released by the frontend team
In progress	Some decisions or implementation remain
Pending	Not started
Blocked	A required backend contract or product decision is unavailable

Phase Summary

Original phase	Frontend scope	Status	Backend dependency
Phase 0	Frontend architecture, UX, and contract decisions	Pending	Stable API and deployment conventions
Phase 1	Frontend repository foundation	Pending	None beyond supported Node/pnpm versions
Phase 6	Frontend authentication	Pending	Backend Phase 5 complete
Phase 7	Organization and membership interfaces	Pending	Backend Phase 7
Phase 8	Permission-aware interface behavior	Pending	Backend Phase 8 complete
Phase 9	Email-link journeys and delivery integration QA	Ready	Backend Phase 9 complete
Phase 10	Billing and entitlement interfaces	Ready	Backend Phase 10 complete
Phase 11	Dashboard and settings experience	Pending	Stable backend feature APIs
Phase 12	Notification interface	Ready	Backend Phase 12 complete
Phase 13	File-management interface	Pending	Backend Phase 13
Phase 14	Frontend production hardening	Pending	Stable deployment architecture
Phase 15	Frontend CI/CD, testing, and documentation	Pending	Stable frontend scope and API contract
Release	Demo, beta UX, packaging, and launch assets	Pending	Backend and frontend release candidates

Phase 0: Frontend Architecture and Product Decisions

Status: Pending

Objective: Establish frontend conventions before feature screens are built.

Decisions Already Supplied by the Backend

1. Use the versioned REST API under `/api/v1`.
2. Use Swagger/OpenAPI as the initial contract source.
3. Keep access tokens in memory instead of localStorage or sessionStorage.
4. Send refresh and logout requests with browser credentials enabled.
5. Treat the refresh token as an HttpOnly cookie that frontend JavaScript cannot read.
6. Allow authenticated users to exist before email verification.
7. Treat backend authorization and tenant scoping as authoritative.
8. Use explicit organization IDs for tenant-scoped routes.

Remaining Frontend Decisions

1. Confirm the frontend repository location, Git remote, branch rules, and release ownership.
2. Confirm the React, Vite, TypeScript, router, styling, component, state, form, and test libraries.
3. Approve browser support and responsive viewport targets.
4. Approve the design-system direction, typography, color tokens, icon library, and theme behavior.
5. Define API base URL and environment conventions for local, preview, staging, and production builds.
6. Decide whether API types are handwritten from OpenAPI or generated and versioned.
7. Define analytics, error monitoring, privacy, consent, and public-route requirements.
8. Define frontend deployment and preview-environment targets.

Recommended Baseline

1. Use React, Vite, and TypeScript.
2. Use React Router for routing.
3. Use Tailwind CSS and shadcn/ui for the component foundation.
4. Use Zustand only for small client-owned state such as in-memory authentication and active organization selection.
5. Use a server-state/query library for API caching, invalidation, and request lifecycle behavior.
6. Use schema-based form validation shared between forms and API error mapping where practical.
7. Use Vitest and React Testing Library for unit/integration tests and Playwright for browser journeys.
8. Use Lucide icons through the selected component system.

Completion Criteria

1. Frontend ADRs record the chosen architecture and design-system baseline.
2. No feature team needs to invent token storage, API base URL, or browser support rules.

Phase 1: Frontend Repository Foundation

Status: Pending

Objective: Create an independently maintainable frontend repository with a repeatable toolchain and development environment.

Ordered Steps

1. Create the `nestship-frontend` Git repository and define team access.
2. Scaffold React, Vite, and TypeScript using the approved Node and pnpm versions.
3. Enable strict TypeScript and consistent import/path aliases.
4. Add formatting, linting, type-checking, unit-test, browser-test, build, preview, and development scripts.
5. Add Tailwind CSS, shadcn/ui, design tokens, and the approved icon library.
6. Add React Router and the application route layout.
7. Add environment validation for API URL, public frontend URL, and feature flags.
8. Add an API client that supports JSON, standard backend errors, request IDs, bearer access tokens, and credentialed cookie requests.
9. Add test utilities, mocked API handlers, and an initial render/build smoke test.
10. Add `.env.example`, setup documentation, line-ending policy, and secret-handling guidance.
11. Configure the local frontend origin to match the backend CORS allowlist.

Deliverables

1. A working local application that can call backend liveness and readiness endpoints.
2. A reusable page shell and route-error boundary without marketing-page placeholder content.
3. A clean install, type check, test, and production build.

Completion Criteria

1. A clean checkout runs independently from the backend repository.
2. Local configuration contains no committed credentials.
3. The frontend can display a controlled result from the backend API.

Phase 6: Frontend Authentication

Status: Pending

Objective: Deliver the first complete browser journey against the completed backend authentication API.

Backend Contracts Available

1. Registration and local login.
2. Access-token refresh through an HttpOnly cookie.
3. Current-user retrieval.
4. Current-session logout and logout-all.
5. Email verification request and confirmation.
6. Forgot-password and reset-password flows.
7. Optional Google and GitHub OAuth entry points.
8. Standard validation, authentication, conflict, throttle, and server-error responses.

Ordered Steps

1. Define public, authentication, verification, and protected route groups.
2. Build a centralized authentication store that keeps the access token in memory only.
3. Build an API interceptor or request wrapper that attaches the bearer access token.
4. Implement one guarded refresh attempt when an authenticated request returns `401`.
5. Prevent refresh loops and coordinate simultaneous failed requests through one refresh operation.
6. Restore a browser session on application startup by calling the refresh endpoint with credentials.
7. Implement registration with normalized field handling and backend validation mapping.
8. Implement local login with safe generic credential-error display.
9. Implement current-user loading and protected-route transitions.
10. Implement email-verification status, resend, confirmation, and already-verified states.
11. Implement forgot-password and reset-password forms with generic request confirmation.
12. Implement current-session logout and logout-all behavior.
13. Add Google and GitHub buttons behind frontend configuration flags.
14. Handle the OAuth callback by calling refresh, loading the user, and removing transient URL state.
15. Add loading, empty, disabled, expired-token, throttled, offline, and server-error states.
16. Add accessible labels, focus management, keyboard behavior, and announcement of form errors.
17. Add a protected dashboard placeholder as the first post-login destination.
18. Add unit/integration tests for forms, store behavior, refresh coordination, and protected routes.
19. Add Playwright journeys for register, verify, login, refresh restoration, reset, and logout.

Security Requirements

1. Never persist the access token in browser storage.
2. Never attempt to read or copy the refresh cookie.

3. Use `credentials: 'include'` only for approved API requests.
4. Never place access or refresh tokens in URLs.
5. Do not expose account existence through custom forgot-password messaging.
6. Treat hidden controls as usability only; the backend must still deny unauthorized actions.

Completion Criteria

1. A user can register, verify, log in, restore a session after reload, access a protected route, reset a password, and sign out.
2. Authentication failures do not create refresh loops or stale protected content.
3. Browser tests cover the supported happy path and high-risk failures.

Phase 7: Organization and Membership Interfaces

Status: Pending

Objective: Let multi-organization users create, select, and manage tenant contexts through the backend Phase 7 API.

Ordered Steps

1. Add organization routes keyed by explicit organization ID.
2. Add organization creation with verification-required handling.
3. Add organization list and active-selection behavior.
4. Keep active organization selection in frontend state and a non-sensitive persistence mechanism if approved.
5. Validate persisted selection against the latest membership list on startup.
6. Add organization profile/settings forms.
7. Add invitation creation, list, resend, revoke, expiry, and duplicate states.
8. Add invitation-acceptance routes for authenticated and not-yet-authenticated recipients.
9. Preserve an invitation destination through registration/login without exposing its token unnecessarily.
10. Add member list, role display, removal, voluntary leave, and ownership-transfer interfaces.
11. Require explicit confirmation for ownership transfer, member removal, leaving, and deletion.
12. Handle membership loss or organization deletion while the organization is active.
13. Add pagination, loading, empty, stale, and permission-denied states.
14. Add frontend tests for switching, invitation acceptance, ownership transfer, and membership loss.

Backend Dependencies

1. Organization CRUD contracts.
2. Membership and invitation contracts.
3. Ownership-transfer and organization-deletion semantics.
4. Cross-tenant-safe resource routes.

Completion Criteria

1. A user can switch among organizations without stale tenant data.
2. Invitation and membership workflows reflect backend state accurately.
3. Losing membership immediately removes access to the affected organization UI.

Phase 8: Permission-Aware Interface Behavior

Status: Pending

Objective: Apply the backend capability matrix consistently to frontend navigation, actions, and explanatory states.

Ordered Steps

1. Consume role and capability information from an approved backend contract.
2. Add centralized capability helpers instead of scattered role-name checks.
3. Hide or disable unavailable actions according to the agreed UX convention.
4. Protect organization settings, invitations, member changes, ownership, billing, file, and destructive controls.
5. Display clear permission-denied responses when backend state changes after rendering.
6. Invalidate capability-dependent data after role or membership changes.
7. Add table-driven component and route tests for Owner, Admin, Member, and Viewer.
8. Confirm that no frontend behavior is treated as an authorization boundary.

Completion Criteria

1. Visible actions match the backend capability matrix.
2. Stale permissions fail safely and recover after data refresh.
3. Every restricted frontend action has role-based test coverage.

Phase 9: Email-Link Journeys and Integration QA

Status: Ready

Objective: Complete browser behavior for links emitted by the production transactional email system.

Ordered Steps

1. Confirm public frontend URLs for verification, reset, invitation, and security-notice links.
2. Parse tokens only on the route that consumes them.
3. Remove consumed or invalid tokens from browser history after processing.
4. Add success, expired, already-used, revoked, and malformed-link states.
5. Add resend or restart actions where supported by the backend.
6. Ensure unauthenticated invitation links preserve their destination through login or registration.
7. Test links generated in local, preview, staging, and production environments.
8. Verify responsive and accessible rendering across supported email-to-browser journeys.

Completion Criteria

1. Every production email link reaches the correct frontend route and backend action.
2. Sensitive tokens do not remain in browser history after consumption.
3. Failure states offer a safe recovery path.

Backend Contract Now Available

1. Verification links target `/verify-email?token=...`.
2. Password-reset links target `/reset-password?token=...`.
3. Invitation links target `/invitations/accept?token=...`.
4. Link origins are environment-specific and produced from the backend `FRONTEND_URL`.

Phase 10: Billing and Entitlement Interfaces

Status: Pending

Objective: Let authorized organization users view plans, start checkout, manage subscriptions, and understand entitlements without duplicating billing truth in the browser.

Ordered Steps

1. Consume the approved plan catalog and organization billing summary.
2. Build plan comparison and monthly/annual selection interfaces.
3. Start Stripe Checkout through the backend and redirect only to returned approved URLs.
4. Start Customer Portal sessions through the backend.
5. Add post-checkout success, cancellation, delayed-webhook, and synchronization states.
6. Display trial, active, cancellation-scheduled, past-due, grace, canceled, and incomplete states.
7. Show entitlement-based upgrade prompts without treating them as authorization.
8. Restrict billing controls using backend Phase 8 capabilities.
9. Add loading, retry, provider-unavailable, and stale-status states.
10. Add tests using mocked backend billing contracts and Stripe redirect boundaries.

Completion Criteria

1. Authorized users can subscribe and manage billing through backend-created sessions.
2. The UI accurately reflects local backend billing state.
3. Feature visibility never bypasses backend entitlement enforcement.

Phase 11: Dashboard and Settings Experience

Status: Pending

Objective: Turn completed product modules into a polished, reusable SaaS application interface.

Ordered Steps

1. Build the responsive application shell, sidebar, top bar, and mobile navigation.
2. Add user and organization switchers with predictable focus and keyboard behavior.
3. Add dashboard summary cards, charts, recent activity, and meaningful empty states.
4. Add reusable tables with filtering, sorting, pagination, row actions, and responsive alternatives.
5. Add dialogs, sheets, menus, tooltips, toasts, and confirmations through the design system.
6. Build Profile, Password, Team, Billing, and Notification settings sections.
7. Add onboarding and first-organization flows.
8. Add skeleton, loading, empty, stale, error, and offline states for every data surface.
9. Add light and dark themes through tokens rather than component-specific colors.
10. Audit responsive layouts, text containment, accessibility, focus, and reduced motion.
11. Document reusable frontend patterns separately from NestShip example content.

Completion Criteria

1. Core workflows are efficient on supported desktop and mobile viewports.
2. Settings surfaces are connected to real backend contracts.
3. Reusable components meet documented accessibility and interaction standards.

Phase 12: Notification Interface

Status: Pending

Objective: Present persistent notifications and preferences from the backend Phase 12 API.

Ordered Steps

1. Add a notification menu and full notification page.
2. Add unread count, mark-read, and mark-all-read behavior.
3. Add stable pagination or infinite loading based on the backend contract.
4. Add notification preference controls.
5. Use polling initially at an approved interval and pause when the page is hidden when practical.
6. Add deep links to organization, billing, account, and file destinations.
7. Handle removed destinations, expired notifications, and lost tenant access.
8. Add loading, empty, offline, and retry states.
9. Add tests for unread synchronization, pagination, preferences, and tenant switching.

Completion Criteria

1. Users can view and manage notifications without duplicate or stale counts.
2. Notification links respect current membership and permissions.

Phase 13: File-Management Interface

Status: Pending

Objective: Provide safe organization-scoped file workflows over the backend storage API.

Ordered Steps

1. Add upload controls with approved type and size guidance.
2. Validate files before upload while treating backend validation as authoritative.
3. Add upload progress, cancellation, retry, and failure cleanup messaging.
4. Add organization file listing with pagination and metadata.
5. Retrieve short-lived signed download or preview URLs only when needed.
6. Add permission-aware download and delete actions.
7. Add confirmation and optimistic-state rollback for deletion.
8. Handle expired signed URLs by requesting a fresh URL.
9. Avoid exposing private object keys or provider credentials.
10. Add tests for validation, progress, expired URLs, permission loss, and failure recovery.

Completion Criteria

1. Authorized users can upload, retrieve, and delete files through backend contracts.
2. Private provider details and permanent URLs never enter client code.

Phase 14: Frontend Production Hardening

Status: Pending

Objective: Meet measurable browser security, accessibility, performance, compatibility, and deployment standards.

Ordered Steps

1. Add route-level error boundaries and production error monitoring.
2. Add source-map, release, and environment tagging policies.
3. Review token handling, credentialed requests, redirects, URL tokens, and sensitive logging.
4. Add Content Security Policy compatibility and remove unsafe inline assumptions.
5. Measure and improve initial loading, route loading, bundle size, and key interaction performance.
6. Add code splitting where it improves real route behavior.
7. Run accessibility audits and keyboard-only workflow reviews.
8. Run supported-browser and responsive-viewport tests.
9. Validate preview, staging, and production API/CORS/cookie behavior.
10. Add deployment rollback, cache invalidation, and incident guidance.

Completion Criteria

1. Supported user journeys meet approved accessibility and browser targets.
2. Production errors are observable and tied to release versions.
3. Authentication and API integration work in the reference deployment topology.

Phase 15: Frontend CI/CD, Testing, and Documentation

Status: Pending

Objective: Make frontend delivery repeatable, verifiable, and maintainable.

Ordered Steps

1. Add GitHub Actions for frozen install, format check, lint, type check, unit/integration tests, build, and Playwright tests.
2. Add preview deployment checks for pull requests if supported by the deployment provider.
3. Add regression coverage for authentication restoration, invitation flows, organization switching, capability changes, billing redirects, notifications, and files.
4. Add visual regression coverage for high-value responsive views if approved.
5. Version or generate API types from the approved OpenAPI contract.
6. Document setup, environments, architecture, state ownership, API usage, testing, deployment, accessibility, and extension patterns.
7. Define frontend release tags, changelog, browser support, backend compatibility, and upgrade policies.
8. Add dependency-update automation with required browser tests.

Completion Criteria

1. Every pull request and release runs the required frontend checks automatically.
2. A new frontend developer can configure, test, deploy, and extend the application from documentation.
3. Frontend releases declare compatible backend API versions.

Release: Demo, Beta, Packaging, and Launch

Status: Pending

Objective: Present and validate the complete NestShip experience for beta users and commercial buyers.

Ordered Steps

1. Build a realistic example application using only supported extension points.
2. Prepare screenshots and product demonstrations from real working states.
3. Add a controlled demo environment with non-sensitive resettable data.
4. Support beta onboarding and collect structured usability feedback.
5. Resolve release-blocking accessibility, responsive, browser, and workflow defects.
6. Prepare product landing, documentation, repository presentation, and launch assets.
7. Publish versioned frontend release notes and backend compatibility guidance.

Completion Criteria

1. Beta users can complete the core account, organization, billing, notification, and file journeys.

2. Product media accurately represents the shipped application.
3. The frontend release is versioned, documented, and compatible with the backend release.

Frontend-to-Backend Contract Rules

1. Consume `/api/v1` and the published OpenAPI contract rather than undocumented endpoints.
2. Send the standard bearer access token only from in-memory auth state.
3. Use credentialed requests for refresh and cookie-backed logout endpoints.
4. Map the backend standard error response, including request ID, into user-safe messages and support diagnostics.
5. Use explicit organization IDs for tenant-scoped navigation and API calls.
6. Refetch membership, capabilities, and entitlements after mutations that can change access.
7. Never infer permission or billing truth solely from hidden frontend controls.
8. Coordinate breaking contract changes through versioned releases and compatibility notes.

Initial Backend Handoff Available to the Frontend Team

1. Backend repository on `main` with backend Phases 1 through 5 and Phases 7 through 12 complete.
2. Local API default at `http://localhost:3000/api/v1`.
3. Swagger UI at `http://localhost:3000/api/docs`.
4. OpenAPI JSON at `http://localhost:3000/api/docs-json`.
5. Local frontend CORS origin at `http://localhost:5173`.
6. Completed authentication endpoints and refresh-cookie semantics.
7. Development-only verification and reset links in backend debug logs.
8. Google and GitHub OAuth endpoints that return `503` until provider credentials are configured.
9. Organization, invitation, membership, ownership-transfer, pagination, and tenant-isolation contracts.
10. Effective `currentUserCapabilities` values and the ADR 0006 authorization matrix.

Immediate Next Frontend Task

Complete Phase 0 frontend decisions, create the `nestship-frontend` repository, and execute Phase 1 repository foundation before beginning the Phase 6 authentication interface.